

	Année Universitaire : 2025/2026
	Module : Système d'exploitation II
	Classe : CPI2
	Correction TD : Synchronisation interprocessus

Exercice 1 :

Pour mieux comprendre cet exercice, nous pouvons prendre l'exemple d'un enseignant (Processus Maître) avec ses N étudiants (Processus P_i). L'enseignant peut donner un exercice **PA** à faire et entre temps il fait l'appel **MA**. Il ne corrige l'exercice **MB** que si tous les étudiants ont le fini. Une fois il termine la correction **MB**, il permet aux étudiants de passer à un autre exercice **PB**.

Ainsi MB ne peut jamais être exécuté avant la fin de PA. Ce problème de coordination peut facilement être résolu en utilisant un sémaphore S2 qui est initialisé avec la valeur 0. Ceci permet de bloquer MB tant que les N étudiants n'ont pas fini PA.

On a besoin d'une variable i pour compter les N processus P_i .

Correction

Déclarations : S1 : semaphore (init à 0); S2 : semaphore (init à 0); mutex : semaphore (init à 1); i : entier (init à 0); N constante représentant le nombre de processus P_i .

<u>Processus P_i</u> Début PA ; P(mutex) i := i + 1 ; V(mutex) Si i = N alors V(S2) ; Finsi P(S1) ; PB ; V(S1); Fin	<u>Processus Maître</u> Début MA ; P(S2) ; MB ; V(S1) ; Fin
--	--

Code commenté

<u>Processus P_i</u> Début PA ; P(mutex) ; /* mutex est utilisé pour protéger la variable i qui est partagé entre les processus P_i */ i := i + 1 ; /* si un processus exécute PA alors on incrémente i */ V(mutex) ; /* libérer la variable i pour qu'un autre processus peut entrer */ Si i = N alors V(S2) ; /* Si N processus terminent l'exécution de PA alors on réveille le processus maître pour exécuter MB */ Finsi P(S1) ; /* S1 est initialisé à 0 donc les processus P_i ne peut pas exécuter PB en attendant que le processus maître envoie un signal V(S1) */ PB ;	<u>Processus Maître</u> Début MA ; /* le processus maître exécute MA en parallèle avec l'instruction PA et se bloque en attendant que les N processus P_i terminent l'exécution de l'instruction PA */ P(S2) ; /* S2 est initialisé à 0 donc le processus Maître est bloqué pour ne pas exécuter MB en attendant que le N ^{ième} processus incrémente la valeur du sémaphore S2 */ MB ; V(S1) ; /* le processus maître réveille un processus P_i pour exécuter la fonction PB */ Fin
--	--

V(S1) ; /* PB est une section critique pour les N processus Pi donc il faut toujours incrémenter la valeur de S1 pour que le prochain processus exécute aussi PB. Si on enlève cette instruction, on aura un seul processus qui peut exécuter PB*/ Fin	
---	--

Exercice 2 :

C'est le même exemple étudié dans le cours (vidéo) sauf que cette correction implémente les procédures *déposer ()* et *retirer()*.

Le producteur dépose les messages en queue et le Consommateur prend les messages de la tête du tampon.

On déclare principalement :

- un tableau de N messages nommé Tampon
- deux variables Queue et tête initialisées à 0 une pour l'indice de déposer et l'autre pour l'indice de retirer

Les procédures *retirer* et *déposer* suivent la même logique. Donc, nous expliquons seulement la procédure *déposer*. Cette dernière permet d'insérer le message MessProd dans le tampon à la position Queue et passe Queue à la position suivante. Cette position est donnée par le reste de la division de la valeur de queue plus 1 par la taille du tampon ($Queue = (Queue + 1) \bmod N$). Ceci garantit que si on arrive à la position N-1 (dernier élément du tampon) et que le premier élément est retiré du tampon alors la position suivante d'insertion est $0 = (N - 1 + 1) \bmod N$ comme dans le cas suivant :

		Message	Message	Message	Message	Message
--	--	---------	---------	---------	---------	---------

Contexte commun: SnPlein: **sémaphore** initialisé à 0; SnVide: **sémaphore** initialisé à N;

Tampon: Tableau de N messages;

<u>Processus Producteur</u>	<u>Processus Consommateur</u>
Queue: <u>entier</u> initialisé à 0;	Tete: <u>entier</u> initialisé à 0;
MessProd: message;	MessCons: message;
<u>Début</u>	<u>Début</u>
<u>Tant que vrai faire</u>	<u>Tant que vrai faire</u>
Fabriquer(MessProd);	P(SnPlein);
P(SnVide);	MessCons= Tampon[Tete];
Tampon [Queue]=MessProd ;	Tete=(Tete +1) mod N ;
Queue=(Queue +1) mod N ;	V(SnVide);
V(SnPlein);	Traiter (MessCons);
<u>Fait ;</u>	<u>Fait ;</u>
<u>Fin</u>	<u>Fin</u>

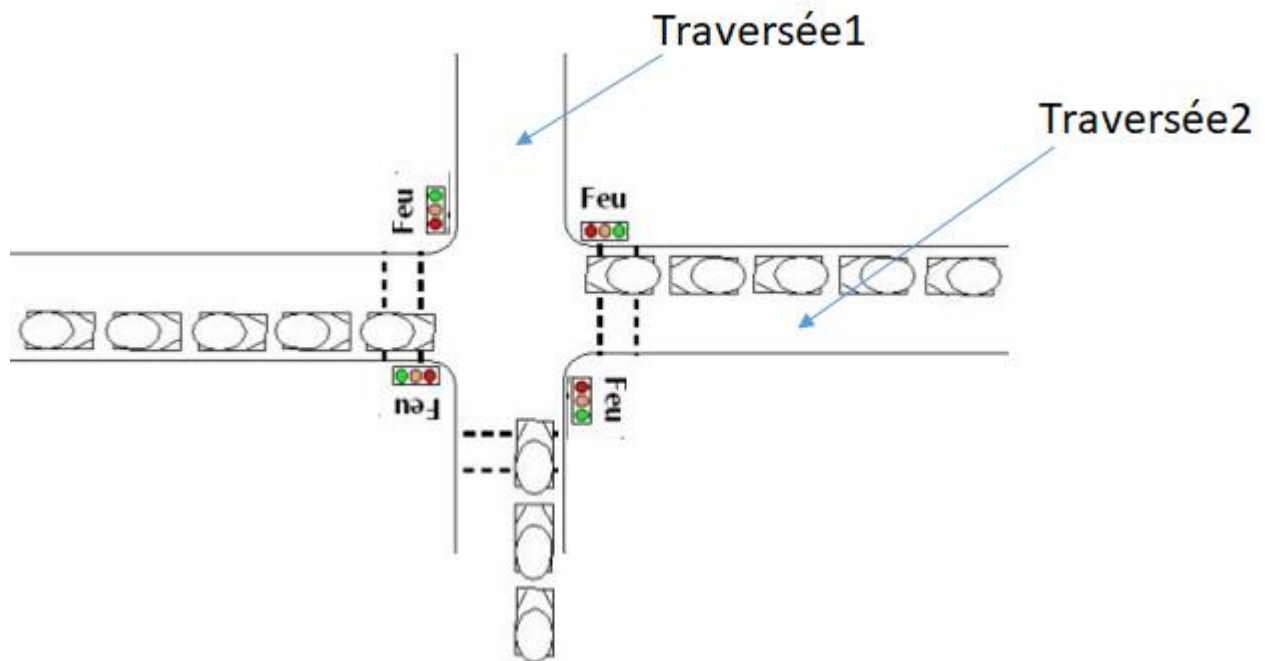
Exercice 3

<u>Programme client</u> <pre>Client() { P(SX); // verrouiller l'accès à Attend if(Attend < N) { Attend = Attend + 1; V(SP); // signaler au coiffeur qu'un client attend V(SX); // libérer le mutex P(SCF); // attendre que le coiffeur l'appelle SeFaireCoifferEtSortir(); } else { V(SX); // salle pleine → libérer le mutex Sortir(); } }</pre>	<u>Fonctionnement du Client</u> 1. incrémente Attend (accès exclusif) 2. Si le nombre de clients en attente est supérieur à N, sort 3. Sinon, s'ajoute à la file d'attente 4. attend de pouvoir s'asseoir dans le siège du coiffeur 5. se fait coiffer et sort.
<u>Programme coiffeur</u> <pre>Coiffeur(){ while (1){ P(SP); // attendre un client (le coiffeur dort ici) P(SX); // verrouiller Attend Attend = Attend -1; V(SCF); // appeler un client pour entrer dans le fauteuil V(SX); // relâcher l'accès à Attend Coiffer(); } }</pre>	<u>Fonctionnement du Coiffeur</u> 1. prend le 1er client de la file, s'il y en a un (sinon il se bloque) 2. décrémente Attend (accès exclusif) 3. libère le siège du client (l'invite à s'asseoir) 4. Coiffe le client

2- Rôle de chacun des sémaphores SCF, SP et SX :

1. SX est un sémaphore mutex qui garantit l'accès exclusif à la variable partagée **Attend**, afin de maintenir correctement le comptage des clients dans la salle d'attente.
2. SP est un sémaphore compteur utilisé par les clients pour signaler leur présence ; il indique au coiffeur combien de clients attendent et le réveille lorsqu'au moins un arrive.
3. SCF est un sémaphore de synchronisation par lequel le coiffeur appelle un client : un client ne peut s'asseoir dans le fauteuil que lorsque le coiffeur a émis un signal via ce sémaphore.

Exercice 4



Le fonctionnement d'un système de gestion des feux de circulation d'un carrefour peut être modélisé par un ensemble de processus parallèles : Changement, Traversee1 et Traversee2.

L'arrivée des voitures est aléatoire. Une voiture qui veut traverser le carrefour sur voie1 exécute Traversee1 et celle qui veut traverser sur voie2 exécute Traversee2. Ces deux procédures font appel à une fonction Traversee() qui simule la traversée de l'intersection.

Processus Traversee1

Puisque le carrefour peut contenir au plus une voiture à la fois donc même si on a plus qu'une voiture sur une voie et que le feu de cette voie est vert alors la traversée du carrefour ne se fait que voiture par voiture. Par conséquent, la fonction traversee() sur une voie est une section critique. D'où, un sémaphore mutex SX1 est nécessaire pour protéger cette section pour toutes les voitures de voie 1.

De plus, un changement de feu ne peut se faire que s'il n'y a pas de voiture au carrefour. Donc, on doit bloquer le changement de feu si une voiture est entrain de traverser le carrefour et la durée du feu s'achève et bloquer aussi la traversée d'une voiture sur une voie si le feu de cette voie doit être changé. Ainsi, on doit utiliser un sémaphore SF1 pour assurer la synchronisation entre les processus Traversee1 et Changement.

Si une voiture de la voie 1 est dans le carrefour (Exécute traversee()) et la durée de feu est achevée donc le changement de feu est bloqué par P(SF1).

Processus Traversee 2

De la même façon que la Traversee1, on a utilisé un sémaphore binaire SX2 pour protéger la fonction traversee() et un autre sémaphore SF2 pour synchroniser les processus Changement et Traversee2.

Processus Changement

Le principe de ce processus est le changement de feu, du vert au rouge ou inversement, entre les deux traversées chaque période (durée du feu) sachant qu'un changement ne peut se faire que s'il n'y a pas de voiture au carrefour. Par exemple, si le feu est vert dans la traversée 1 (feu==1) et il n'y a pas une voiture dans le carrefour alors ce processus bloque l'accès au carrefour à partir de traversee 1 (P(SF1)) et réveille le processus Traversee2 (V(SF2)) et le feu de la voie 2 sera vert (feu=2). Sinon (si feu==2) alors il bloque la traversée 2 avec P(SF2) et réveille la Traversee1 avec V(SF1) et le feu de la traversée 1 sera vert (feu=1).

Init(SX1, 1) , Init(SX2, 1), Init(SF1, 1), Init(SF2, 0);	
<pre>Changement { int Feu=1; //initialement Feu vert sur voie1 while(1){ sleep(Duree_du_feu); if(Feu==1){ //Feu vert sur voie1 P(SF1); //Bloquer traversée sur voie 1 V(SF2); //Réveiller voiture sur voie 2 Feu = 2; //Mettre le feu vert sur voie 2 } else //Feu vert sur Voie2 { P(SF2); // Bloquer traversée sur voie 2 V(SF1); //Réveiller voiture sur voie 1 Feu=1; //Mettre le feu vert sur voie 1 } } }</pre>	
<pre>Traversee1 //Traverser voie 1 { P(SX1); //Protéger une section critique Traversee() P(SF1); // Attendre feu vert voie 1 Traversee(); // Simule la traversée de l'intersection V(SX1); /*Débloquer la traversée pour que la prochaine voiture peut traverser le carrefour*/ }</pre>	<pre>Traversee2 { P(SX2); P(SF2); Traversee(); V(SF2); V(SX2); }</pre>